

CSCE 410 P1 Report: Logging Functionality

Zayd Nadir

1. Submission information

a. Repository URL: <https://github.com/zaydn4321/333006458-xv6-csce410-p1>

b. Late-day tokens: No, none used.

2. Resources used

a. Which resources did you leverage while developing your solution?

I used Claude Code along with the assignment handout and the existing xv6 source code. I wrote the main parts of the code myself, including the `dprintf` macro, the `debugctl` system call, the `dbg` tool, and the instrumentation in `proc.c` and `exec.c`. Some parts were generated by Claude Code.

b. How were they useful?

I used Claude Code to clarify parts of the assignment and instructions I wasn't sure about before starting. When I had it generate code, I told it exactly what I wanted and it wrote it the way I asked. I also used it for syntax (C macros, GCC attributes, the `__atomic` builtins, Makefile changes) and for debugging when something didn't compile or behaved differently than I expected. The simulations for the demonstration were done with the help of Claude Code. It ran the test scenarios in xv6 and saved the transcripts. This report was also written with the help of Claude Code. I reviewed and tested everything it produced, and it was a tool I directed rather than the main source of the work.

c. What is your estimate for how much time these resources saved you?

About 8 to 10 hours. Most of that came from not having to search documentation and forums for syntax details and compiler errors, and from getting unstuck faster while debugging.

d. If you had to redo the assignment without any of the resources, how many hours do you think it would take you?

About 20 to 25 hours. The design wouldn't change, but I would have spent much longer looking up syntax, reading documentation on things like GCC atomics and xv6's build system, and tracking down bugs on my own. Writing the report and putting together the transcripts would also have taken longer.

3. Source code added or modified

New files:

- kernel/debug.h - categories, levels, debugctl ops, the name table, and the dprintf macro
- kernel/debug.c - the mask, level and pid variables, and sys_debugctl
- user/dbg.c - the dbg program

Modified files:

- Makefile - added debug.o, _dbg, and the DBG=0 option
- kernel/syscall.h, kernel/syscall.c - added the debugctl syscall (number 23)
- user/usys.pl, user/user.h - user side of debugctl
- kernel/proc.c - log messages in allocproc, freeproc, userinit, kfork, kexit, kwait, kkill and scheduler. reparent() now returns how many children it moved.
- kernel/exec.c - log messages in kexec

4. Transcripts (console output) included

All in transcripts/. Each file is a full xv6 session with the commands and output.

- 1_boot_silent.txt - scenario 1. Boot with mask 0, no debug output.
- 2_proc_ls_echo.txt - scenario 2. dbg on proc, then ls and echo hi. Shows fork, exec, exit and wait with matching pids.
- 3_forktest.txt - scenario 3. forktest at level trace.
- 3_forktest_check.txt - checks that every pid in forktest exited and was freed once. No leaks or double frees.
- 4_off_all.txt - scenario 4. ls, echo hi and forktest with logging on, then dbg off all and the same commands with no output.
- 5_sched.txt - scenario 5. dbg on sched and what it prints.
- 6_usertests.txt - scenario 6. usertests passes with mask 0.
- 7_bonus_pid_filter.txt - the bonus.
- 8_extra_errors_levels.txt - extra tests: kill of a missing pid, exec of a missing program, different levels, and bad dbg input.

5. Design comments

I followed the design from parts A to D.

dprintf. A message prints if its category is on in the mask, its level is at or below the current level, and (for the bonus) the pid filter matches. Each message starts with the category, pid and function name, like [proc][2][kfork] ..., so grep '\[proc ' finds all the process messages. The pid is -1 when there is no current process, like in userinit(). The names are padded by hand because printf doesn't support %-7s.

It is a macro so the arguments only get evaluated when it prints, and so __func__ is the caller's name. make DBG=0 turns it into if (0) printf(...), which the compiler removes. I used if (0) instead of

nothing so variables that are only logged don't cause unused variable errors with `-Werror`. I checked that the `DBG=0` kernel builds with no warnings and that the log strings are not in the binary.

The category names are in one table in `debug.h`. The kernel and `dbg` both use it, so a new category only has to be added once.

debugctl. I used the suggested interface, one syscall with GET/SET ops for the mask and level. SET returns the old value and a bad op or level returns -1. One problem is that -1 is the same as `DBG_ALL`, so an error and “all categories on” look the same.

dbg. Supports `dbg`, `on`, `off`, `only`, `level` and `all` like the handout. Bad input prints an error and doesn't call the kernel.

Levels. `ERR` is for allocation and exec failures. `WARN` is for killing a pid that doesn't exist. `INFO` is for `fork`, `exec`, `exit`, `wait` and `kill`. `TRACE` is for `allocproc`, `freeproc` and the scheduler message.

Making the sched message usable. In `5_sched.txt`, `dbg` on `sched` alone prints nothing because the message is at `TRACE`. At trace level, one `forktest` printed 106 `sched` lines for 2 lines of real output, one for every context switch. To make it usable I would use it with the pid filter, count switches and print a summary instead of every switch, or save the events in a buffer and print them later.

Build. The new code passes `clang-format`, which the repo's GitHub workflow checks. The `RWX` linker warnings were already there in the original tree.

6. Concurrency

I didn't use a lock. The mask, level and pid are each one aligned variable, so a read or write is a single instruction and can't see half of an update. I used relaxed `__atomic` loads and stores so the compiler doesn't cache old values. The worst case is a message printed or skipped right when the setting changes, which is fine for logging. A lock would slow down every CPU for a value that almost never changes, and `printk` already takes its own lock.

One small issue is that two `dbg` commands at the exact same time could lose an update, since `dbg` reads the mask and writes it back. That's fine since `dbg` is run by hand.

7. Bonus points

a. Yes, I submitted the survey.

b. Yes, filter by pid.

i. I added `DBGCTL_GETPID` and `DBGCTL_SETPID` and a `dbg_pid` variable, where 0 means no filter. A message only prints if it comes from that pid. The `dbg` program has `dbg pid <n>` and `dbg pid off`. The filter uses the process that is running, so “parent 2 reaped child 5” shows under pid 2, not pid 5.

ii. `transcripts/7_bonus_pid_filter.txt`. With `dbg pid 2`, `echo hi` only shows the shell's fork and wait. After `dbg pid off`, all of the messages show again.